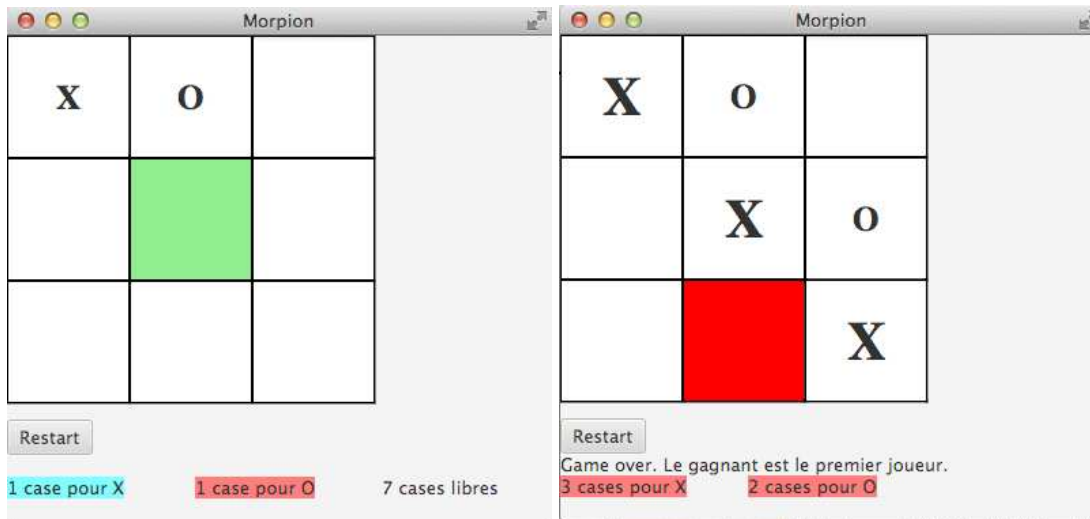


Java – Jeu du Morpion



En se basant sur la classe `TicTacToeModel` développée en TP1, construire un jeu de morpions pour 2 joueurs avec les fonctionnalités suivantes :

- afficher la grille du jeu,
- afficher un bouton “Restart” pour commencer une nouvelle partie,
- afficher le nombres de cases occupées par X,
- afficher le nombres de cases occupées par O,
- afficher le nombres de cases libres (visible seulement si le jeu n’est pas terminé),
- quand le jeu est terminé, afficher le nom du gagnant (ou match nul),
- afficher sur fond bleu clair (cyan) le nombre de cases occupées par le joueur courant, et sur fond rouge pour l’autre joueur,
- afficher la lettre correspondante au joueur courant dans la case cliquée,
- empêcher de rejouer dans une case déjà utilisée,
- quand la souris est sur une case, afficher cette case avec fond vert s’il est possible de jouer dedans, et avec fond rouge sinon (après la fin du jeu, toutes les cases s’afficheront avec fond rouge),
- à la fin du jeu, afficher la (ou les) combinaisons gagnantes avec une police plus grande.

Toutes les mises à jour doivent être faites en utilisant les “bindings”, à l’exception de la couleur de fond sur le passage de la souris. Donc il n’y a que 3 écouteurs sur chaque case (`TicTacToeSquare`, voir plus bas) :

- `setOnMouseClicked`,
- `setOnMouseEntered`,
- `setOnMouseExited`.

La grille du jeu doit être rempli dans le fichier java, pour que cela puisse être généralisé à des grilles de taille plus grande.

```
1 public enum Owner { NONE, FIRST, SECOND;
2     public Owner opposite() {
3         return this == SECOND ? FIRST : this == FIRST ? SECOND : NONE;
4     }
5 }
```

```
1 public class TicTacToeModel {
2     /**
3      * Taille du plateau de jeu (pour être extensible).
4      */
5     private final static int BOARD_WIDTH = 3;
6     private final static int BOARD_HEIGHT = 3;
7     /**
8      * Nombre de pièces alignés pour gagner (idem).
9      */
10    private final static int WINNING_COUNT = 3;
11    /**
12     * Joueur courant.
13     */
14    private final ObjectProperty<Owner> turn =
15        new SimpleObjectProperty<>(Owner.FIRST);
16    /**
17     * Vainqueur du jeu, NONE si pas de vainqueur.
18     */
19    private final ObjectProperty<Owner> winner =
20        new SimpleObjectProperty<>(Owner.NONE);
21    /**
22     * Plateau de jeu.
23     */
24    private final ObjectProperty<Owner>[][] board;
25    /**
26     * Positions gagnantes.
27     */
28    private final BooleanProperty[][] winningBoard;
29
30    /**
31     * Constructeur privé.
32     */
33    private TicTacToeModel() { ... }
34
35    /**
36     * @return la seule instance possible du jeu.
37     */
38    public static TicTacToeModel getInstance() {
```

```
39         return TicTacToeModelHolder.INSTANCE;
40     }
41
42     /**
43      * Classe interne selon le pattern singleton.
44      */
45     private static class TicTacToeModelHolder {
46         private static final TicTacToeModel INSTANCE =
47             new TicTacToeModel();
48     }
49
50     public void restart() { ... }
51
52     public final ObjectProperty<Owner> turnProperty() { ... }
53
54     public final ObjectProperty<Owner> getSquare(int row, int column)
55         { ... }
56
57     public final BooleanProperty getWinningSquare(int row, int column)
58         { ... }
59
60     /**
61      * Cette fonction ne doit donner le bon résultat que si le jeu
62      * est terminé. L'affichage peut être caché avant la fin du jeu.
63      *
64      * @return résultat du jeu sous forme de texte
65      */
66     public final StringExpression getEndOfGameMessage() { ... }
67
68     public void setWinner(Owner winner) { ... }
69
70     public boolean validSquare(int row, int column) { ... }
71
72     public void nextPlayer() { ... }
73
74     /**
75      * Jouer dans la case (row, column) quand c'est possible.
76      */
77     public void play(int row, int column) { ... }
78
79     /**
80      * @return true s'il est possible de jouer dans la case
81      * c'est-à-dire la case est libre et le jeu n'est pas terminé
82      */
83     public BooleanBinding legalMove(int row, int column) { ... }
```

```
84
85     public NumberExpression getScore(Owner owner) { ... }
86
87     /**
88      * @return true si le jeu est terminé
89      * (soit un joueur a gagné, soit il n'y a plus de cases à jouer)
90      */
91     public BooleanBinding gameOver() { ... }
92
93 }
```

```
1 public class TicTacToeSquare extends TextField {
2
3     private static TicTacToeModel model = TicTacToeModel.getInstance();
4
5     private ObjectProperty<Owner> ownerProperty =
6         new SimpleObjectProperty<>(Owner.NONE);
7
8     private BooleanProperty winnerProperty =
9         new SimpleBooleanProperty(false);
10
11     public ObjectProperty<Owner> ownerProperty();
12
13     public BooleanProperty colorProperty();
14
15     public TicTacToeSquare(final int row, final int column) { ... }
16 }
```

Les packages doivent **impérativement** s'appeler `nometudiant1.nometudiant2.morpion` et il faut m'envoyer l'archive zippée du répertoire `src` du projet.